



Institut Supérieur d'Informatique et des Techniques de Communication
de Hammam Sousse

Algorithmique et Structures de données

1^{ère} année LM

Nadra BEN ROMDHANE

Hela FEHRI

nadra.benromdhane@isitc.u-sousse.tn

Algorithmique et Structures de données

Chapitre 1 : Instructions de base Structures conditionnelles Structures répétitives

Notion de variable

- Dans les langages de programmation une **variable** sert à stocker la valeur d'une donnée
- Une variable désigne en fait un emplacement mémoire dont le contenu peut changer au cours d'un programme (d'où le nom variable)
- Règle : Les variables doivent être **déclarées** avant d'être utilisées, elles doivent être caractérisées par :
 - un nom (**Identificateur**)
 - un **type** (entier, réel, caractère, chaîne de caractères, ...)

Choix des identificateurs (1)

Le choix des noms de variables est soumis à quelques règles qui varient selon le langage, mais en général:

- Un nom doit commencer par une lettre alphabétique
exemple valide: A1 **exemple invalide: 1A**
- doit être constitué uniquement de lettres, de chiffres et du soulignement _ (Eviter les caractères de ponctuation et les espaces)
valides: IAG2016, IAG_2016 **invalides: IAG 2016, SI-2016, IAG;2016**
- doit être différent des mots réservés du langage (par exemple en Java: **int, float, else, switch, case, default, for, main, return, ...**)
- La longueur du nom doit être inférieure à la taille maximale spécifiée par le langage utilisé

Choix des identificateurs (2)

Conseil: pour la lisibilité du code, choisir des noms significatifs qui décrivent les données manipulées

exemples: `totalVentes`, `prix_ttc`, `prix_ht`

Remarque: en pseudo-code algorithmique, on va respecter les règles citées, même si on est libre dans la syntaxe

Types des variables

Le type d'une variable détermine l'ensemble des valeurs qu'elle peut prendre, les types offerts par la plupart des langages sont:

- Type numérique (entier ou réel)
 - **Entier court** (codé sur 2 octets) : -32 768 à 32 767
 - **Entier long** (codé sur 4 ou 8 octets)
 - **Réel simple précision** (codé sur 4 octets)
 - **Réel double précision** (codé sur 8 octets)
- Type logique ou booléen: deux valeurs VRAI ou FAUX
- Type caractère: lettres majuscules, minuscules, chiffres, symboles, ...
exemples: 'A', 'a', '1', '?', ...
- Type chaîne de caractère: toute suite de caractères,
exemples: " Nom, Prénom", "code postale: 1000", ...

Déclaration des variables

- Rappel: toute variable utilisée dans un programme doit avoir fait l'objet d'une déclaration préalable
- En pseudo-code, on va adopter la forme suivante pour la déclaration de variables

Variables liste d'identificateurs : type

- Exemple:

Variables i, j, k : entier

x, y : réel

OK: booléen

ch1, ch2 : chaîne de caractères

- Remarque: pour le type numérique, on va se limiter aux entiers et réels sans considérer les sous types

L'instruction d'affectation

- **l'affectation** consiste à attribuer une valeur à une variable (ça consiste en fait à remplir ou à modifier le contenu d'une zone mémoire)
- En pseudo-code, l'affectation se note avec le signe $\leftarrow$
Var $\leftarrow$ e: attribue la valeur de e à la variable Var
 - e peut être une valeur, une autre variable ou une expression
 - Var et e doivent être de même type ou de types compatibles
 - l'affectation ne modifie que ce qui est à gauche de la flèche
- **Ex valides:**

$i \leftarrow 1$	$j \leftarrow i$	$k \leftarrow i+j$
$x \leftarrow 10.3$	$OK \leftarrow FAUX$	$ch1 \leftarrow "IAG"$
$ch2 \leftarrow ch1$	$x \leftarrow 4$	$x \leftarrow j$

(voir la déclaration des variables dans le transparent précédent)
- **non valides:** **$i \leftarrow 10.3$** **$OK \leftarrow "ISITC"$** **$j \leftarrow x$**

Quelques remarques

- Beaucoup de langages de programmation (C/C++, Java, ...) utilisent le signe égal = pour l'affectation $\leftarrow$. Attention aux confusions:
 - l'affectation n'est pas commutative : $A=B$ est différente de $B=A$
 - l'affectation est différente d'une équation mathématique :
 - $A=A+1$ a un sens en langages de programmation
 - $A+1=2$ n'est pas possible en langages de programmation et n'est pas équivalente à $A=1$
- Certains langages donnent des valeurs par défaut aux variables déclarées. Pour éviter tout problème il est préférable **d'initialiser les variables** déclarées

Expressions et opérateurs

- Une **expression** peut être une valeur, une variable ou une opération constituée de variables reliées par des **opérateurs**
exemples: 1, b, a*2, a+ 3*b-c, ...
- L'évaluation de l'expression fournit une valeur unique qui est le résultat de l'opération
- Les **opérateurs** dépendent du type de l'opération, ils peuvent être :
 - des opérateurs arithmétiques: +, -, *, /, % (modulo), ^ (puissance)
 - des opérateurs logiques: NON, OU, ET
 - des opérateurs relationnels: =, ≠, <, >, <=, >=
 - des opérateurs sur les chaînes: & (concaténation)
- Une expression est évaluée de gauche à droite mais en tenant compte de **priorités**

Priorité des opérateurs

- Pour les opérateurs arithmétiques cités , l'ordre de priorité est le suivant (du plus prioritaire au moins prioritaire) :
 - $^$: (élévation à la puissance)
 - $*$, $/$ (multiplication, division)
 - $\%$ (modulo)
 - $+$, $-$ (addition, soustraction)

exemple: $2 + 3 * 7$ vaut 23

- En cas de besoin (ou de doute), on utilise les parenthèses pour indiquer les opérations à effectuer en priorité

exemple: $(2 + 3) * 7$ vaut 35

Les instructions d'entrées-sorties: lecture et écriture (1)

- Les instructions de lecture et d'écriture permettent à la machine de communiquer avec l'utilisateur
- La **lecture** permet d'entrer des données à partir du clavier
 - En pseudo-code, on note: **Lire (var)**
la machine met la valeur entrée au clavier
dans la zone mémoire nommée var
 - Remarque: Le programme s'arrête lorsqu'il rencontre une instruction Lire et ne se poursuit qu'après la frappe d'une valeur au clavier et de la touche Entrée

Les instructions d'entrées-sorties: lecture et écriture (2)

- **L'écriture** permet d'afficher des résultats à l'écran (ou de les écrire dans un fichier)
 - En pseudo-code, on note: **Ecrire (var)**
la machine affiche le contenu de la zone mémoire var
- Conseil: Avant de lire une variable, il est fortement conseillé d'écrire des messages à l'écran, afin de prévenir l'utilisateur de ce qu'il doit frapper

Exemple (lecture et écriture)

Ecrire un algorithme qui demande un nombre entier à l'utilisateur, puis qui calcule et affiche le double de ce nombre

Algorithme Calcul_double

variables A, B : entier

Début

Ecrire("entrer le nombre ")

Lire(A)

$B \leftarrow 2 * A$

Ecrire("le double de ", A, "est :", B)

Fin

Tests: instructions conditionnelles

(1)

- Les instructions conditionnelles servent à n'exécuter une instruction ou une séquence d'instructions que si une condition est vérifiée
- On utilisera la forme suivante: **Si condition alors**
instruction ou suite d'instructions1
Sinon
instruction ou suite d'instructions2
Finsi
 - la condition ne peut être que vraie ou fausse
 - si la condition est vraie, se sont les instructions1 qui seront exécutées
 - si la condition est fausse, se sont les instructions2 qui seront exécutées
 - la condition peut être une condition simple ou une condition composée de plusieurs conditions

Tests: instructions conditionnelles (2)

- La partie Sinon n'est pas obligatoire, quand elle n'existe pas et que la condition est fausse, aucun traitement n'est réalisé
 - On utilisera dans ce cas la forme simplifiée suivante:

Si condition **alors**

instruction ou suite d'instructions1

Finsi

Exemple (Si...Alors...Sinon)

Ecrire un algorithme qui demande un nombre entier à l'utilisateur, puis qui affiche sa valeur absolue.

Algorithme ValeurAbsolue (version1)

Variable x : réel

Début

Ecrire (" Entrez un réel : ")

Lire (x)

Si ($x < 0$) **alors**

Ecrire ("la valeur absolue de ", x, "est:", -x)

Sinon

Ecrire ("la valeur absolue de ", x, "est:", x)

Finsi

Fin

Exemple (Si...Alors)

Algorithme ValeurAbsolue (version2)

Variable x,y : réel

Début

Ecrire (" Entrez un réel : ")

Lire (x)

$y \leftarrow x$

Si ($x < 0$) **alors**

$y \leftarrow -x$

Finsi

Ecrire ("la valeur absolue de ", x, "est:",y)

Fin

Exercice (tests)

Ecrire un algorithme qui demande un nombre entier à l'utilisateur, puis qui teste et affiche s'il est divisible par 3

Algorithme Divisible_par3

Variable n : entier

Début

Ecrire (" Entrez un entier : ")

Lire (n)

Si ($n \% 3 = 0$) **alors**

Ecrire (n, " est divisible par 3")

Sinon

Ecrire (n, " n'est pas divisible par 3")

Finsi

Fin

Conditions composées

- Une condition composée est une condition formée de plusieurs conditions simples reliées par des opérateurs logiques:
ET, OU, OU exclusif (XOR) et NON
- Exemples :
 - x compris entre 2 et 6 : $(x > 2) \text{ ET } (x < 6)$
 - n divisible par 3 ou par 2 : $(n \% 3 = 0) \text{ OU } (n \% 2 = 0)$
- L'évaluation d'une condition composée se fait selon des règles présentées généralement dans ce qu'on appelle tables de vérité

Tables de vérité

C1	C2	C1 ET C2
VRAI	VRAI	VRAI
VRAI	FAUX	FAUX
FAUX	VRAI	FAUX
FAUX	FAUX	FAUX

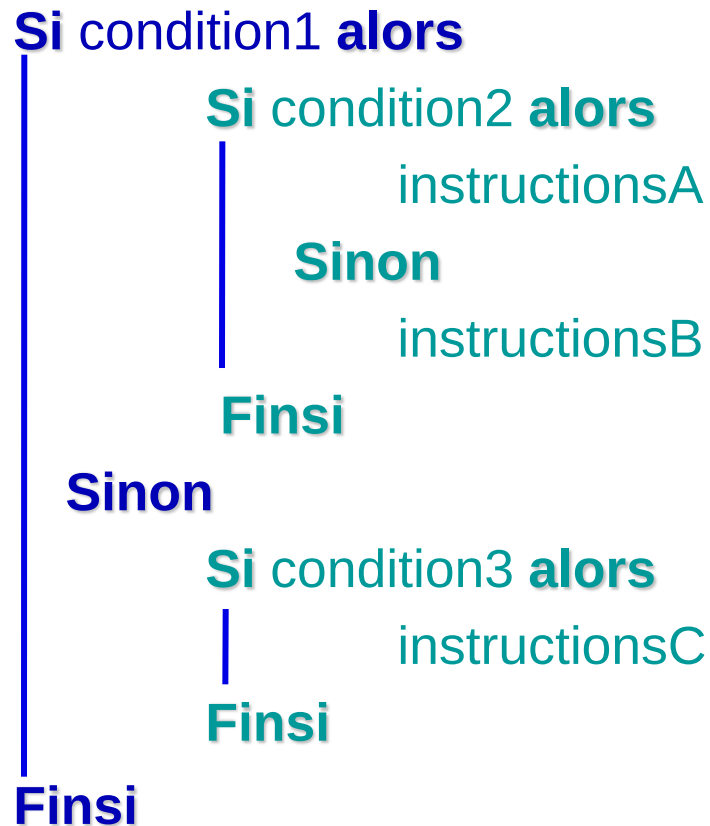
C1	C2	C1 OU C2
VRAI	VRAI	VRAI
VRAI	FAUX	VRAI
FAUX	VRAI	VRAI
FAUX	FAUX	FAUX

C1	C2	C1 XOR C2
VRAI	VRAI	FAUX
VRAI	FAUX	VRAI
FAUX	VRAI	VRAI
FAUX	FAUX	FAUX

C1	NON C1
VRAI	FAUX
FAUX	VRAI

Tests imbriqués

- Les tests peuvent avoir un degré quelconque d'imbrications



Tests imbriqués: exemple (version 1)

Ecrire un algorithme qui demande un nombre entier à l'utilisateur, puis qui teste et affiche s'il est négatif, positif ou nul.

Algorithme PN

Variable n : entier

Début

Ecrire ("entrez un nombre : ")

Lire (n)

Si ($n < 0$) **alors**

 Ecrire ("Ce nombre est négatif")

Sinon

Si ($n = 0$) **alors**

 Ecrire ("Ce nombre est nul")

Sinon

 Ecrire ("Ce nombre est positif")

Finsi

Finsi

Fin

Tests imbriqués: exemple (version 2)

Algorithme PN

Variable n : entier

Début

Ecrire ("entrez un nombre : ")

Lire (n)

Si ($n < 0$) **alors** Ecrire ("Ce nombre est négatif")

Finsi

Si ($n = 0$) **alors** Ecrire ("Ce nombre est nul")

Finsi

Si ($n > 0$) **alors** Ecrire ("Ce nombre est positif")

Finsi

Fin

Remarque : dans la version 2 on fait trois tests systématiquement alors que dans la version 1, si le nombre est négatif on ne fait qu'un seul test

Conseil : utiliser les tests imbriqués pour limiter le nombre de tests et placer d'abord les conditions les plus probables

Structure à choix multiple

Syntaxe générale:

SELON Sélecteur FAIRE

Liste_valeurs1 : **Traitement 1**

Liste_valeurs2 : **Traitement 2**

...

Liste_valeursN : **Traitement N**

Autre : **Traitement**

FIN SELON

Sélecteur : C'est un identificateur de variable ou une expression de type scalaire.

Liste_valeurs1..Liste_valeursN : Ce sont des valeurs servant aux tests. Elles peuvent être données sous forme de constantes et/ou d'intervalles constantes de type compatible avec le sélecteur.

Structure à choix multiple

Syntaxe générale:

SELON num FAIRE

1: Traitement a

2: Traitement b

...

Autre : Traitement X

FIN SELON

SELON num FAIRE

1,3,5 : Traitement a

2,4,6 : Traitement b

...

Autre : Traitement X

FIN SELON

Structure à choix multiple

Syntaxe générale:

SELON car FAIRE

'a' : Traitement a

'b' : Traitement b

...

Autre : Traitement X

FIN SELON

SELON car FAIRE

'a','b','c' : Traitement a

'x','z' : Traitement b

...

Autre : Traitement X

FIN SELON

Structure à choix multiple

Syntaxe générale:

SELON num FAIRE

1 .. 5 : Traitement a

6 .. 10 : Traitement b

...

Autre : Traitement X

FIN SELON

SELON car FAIRE

'a' .. 'h' : Traitement a

'i' .. 'm' : Traitement b

...

Autre : Traitement X

FIN SELON

Exercice

Le prix de tirages dans une librairie varie selon le nombre demandé et la couleur d'impression:

Nombre de copies	Noir et blanc	couleur
[1..10]	150 m	250
[11..20]	120 m	220
> 20	100 m	200

Ecrire un algorithme qui demande à l'utilisateur le nombre de photocopies effectuées et le type de couleur d'impression, et qui calcule et affiche le prix à payer.

Ensuite, l'utilisateur saisie le montant reçu et l'algorithme calcule le montant à rendre.

Enfin, l'algorithme affiche le ticket d'achat.

Exercice

Ecrire un algorithme permettant, à partir d'un menu affiché à l'écran, d'effectuer la somme, le produit ou la moyenne de trois nombres.

Instructions itératives: les boucles

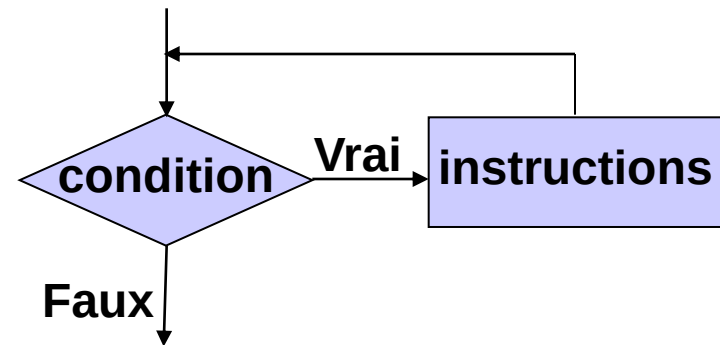
- Les boucles servent à répéter l'exécution d'un groupe d'instructions un certain nombre de fois
- On distingue trois sortes de boucles en langages de programmation :
 - Les **boucles tant que** : on y répète des instructions tant qu'une certaine condition est réalisée
 - Les **boucles jusqu'à** : on y répète des instructions jusqu'à ce qu'une certaine condition soit réalisée
 - Les **boucles pour** ou avec compteur : on y répète des instructions en faisant évoluer un compteur (variable particulière) entre une valeur initiale et une valeur finale

Les boucles Tant que

TantQue (condition)

instructions

FinTantQue



- la condition (dite condition de contrôle de la boucle) est évaluée avant chaque itération
- si la condition est vraie, on exécute instructions (corps de la boucle), puis, on retourne tester la condition. Si elle est encore vraie, on répète l'exécution, ...
- si la condition est fausse, on sort de la boucle et on exécute l'instruction qui est après FinTantQue

Les boucles Tant que : remarques

- Le nombre d'itérations dans une boucle TantQue n'est pas connu au moment d'entrée dans la boucle. Il dépend de l'évolution de la valeur de condition
- Une des instructions du corps de la boucle doit absolument changer la valeur de condition de vrai à faux (après un certain nombre d'itérations), sinon le programme tourne indéfiniment

⇒ **Attention aux boucles infinies**

- Exemple de boucle infinie :

$i \leftarrow 2$

TantQue ($i > 0$)

$i \leftarrow i+1$ (attention aux erreurs de frappe : + au lieu de -)

FinTantQue

Boucle Tant que :

Exercice 1 : Ecrire un algorithme qui contrôle la saisie d'une lettre jusqu'à ce que le caractère entré soit une majuscule.

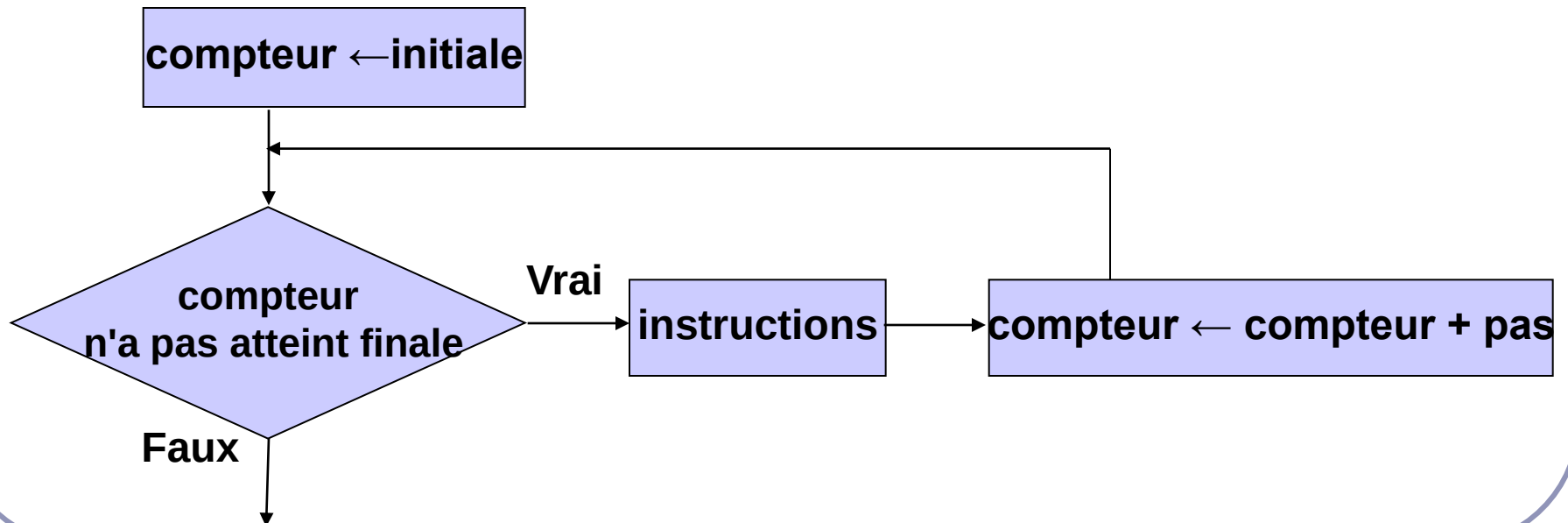
Exercice 2 : Ecrire un algorithme qui détermine le premier nombre entier N tel que la somme de 1 à N dépasse strictement 100

Les boucles Pour

Pour compteur **de** initiale **à** finale par **pas** <valeur du pas>

instructions

FinPour



Les boucles Pour

- Remarque : le nombre d'itérations dans une boucle Pour est connu avant le début de la boucle
- **Compteur** est une variable de type entier (ou caractère). Elle doit être déclarée
- **Pas** est un entier qui peut être positif ou négatif. **Pas** peut ne pas être mentionné, car par défaut sa valeur est égale à 1. Dans ce cas, le nombre d'itérations est égal à finale - initiale + 1
- **Initiale et finale** peuvent être des valeurs, des variables définies avant le début de la boucle ou des expressions de même type que compteur

Déroulement des boucles Pour

- 1) La valeur initiale est affectée à la variable compteur
- 2) On compare la valeur du compteur et la valeur de finale :
 - a) Si la valeur du compteur est $>$ à la valeur finale dans le cas d'un pas positif (ou si compteur est $<$ à finale pour un pas négatif), on sort de la boucle et on continue avec l'instruction qui suit FinPour
 - b) Si compteur est $\leq$ à finale dans le cas d'un pas positif (ou si compteur est $\geq$ à finale pour un pas négatif), instructions seront exécutées
 - i. Ensuite, la valeur de compteur est incrémentée de la valeur du pas si pas est positif (ou décrémenté si pas est négatif)
 - ii. On recommence l'étape 2 : La comparaison entre compteur et finale est de nouveau effectuée, et ainsi de suite ...

Boucle Pour : exemple1

Exercice 3 : Ecrire un algorithme qui permet de calculer x à la puissance n où x est un réel non nul et n un entier positif ou nul

Boucle Pour : remarque

- Il faut éviter de modifier la valeur du compteur (et de finale) à l'intérieur de la boucle. En effet, une telle action :
 - perturbe le nombre d'itérations prévu par la boucle Pour
 - rend difficile la lecture de l'algorithme
 - présente le risque d'aboutir à une boucle infinie

Exemple : **Pour i de 1 à 5 faire**

$i \leftarrow i - 1$

écrire(" i = ", i)

FinPour

Lien entre Pour et TantQue

La boucle Pour est un cas particulier de Tant Que (cas où le nombre d'itérations est connu et fixé) . Tout ce qu'on peut écrire avec Pour peut être remplacé avec TantQue (la réciproque est fausse)

Pour compteur **de** initiale **à** finale par **pas** valeur du pas **faire**
instructions

FinPour

peut être remplacé par :
(cas d'un pas positif)

compteur $\leftarrow$ initiale
TantQue compteur $\leq$ finale
instructions
compteur $\leftarrow$ compteur+pas
FinTantQue

Lien entre Pour et TantQue: exemple

Calcul de x à la puissance n où x est un réel non nul et n un entier positif ou nul (**version avec TantQue**)

Variables x , $puiss$: réel
 n , i : entier

Debut

Ecrire (" Entrez la valeur de x ")
Lire (x)
Ecrire (" Entrez la valeur de n ")
Lire (n)

$puiss \leftarrow 1$

$i \leftarrow 1$

TantQue ($i \leq n$)

$puiss \leftarrow puiss * x$
 $i \leftarrow i + 1$

FinTantQue

Ecrire (x , " à la puissance ", n , " est égal à ", $puiss$)

Fin

Boucles imbriquées

- Les instructions d'une boucle peuvent être des instructions itératives. Dans ce cas, on aboutit à des **boucles imbriquées**

- **Exemple:**

Pour i de 1 à 5 faire

Pour j de 1 à i faire

écrire("message 1")

FinPour

écrire("message 2")

FinPour

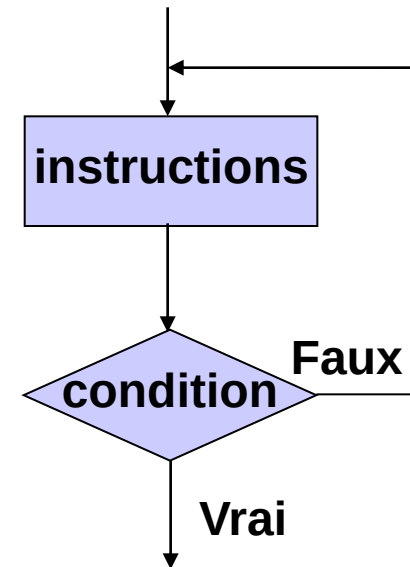
Les boucles Répéter ... jusqu'à

...

Répéter

instructions

Jusqu'à condition



- Condition est évaluée après chaque itération
- les instructions entre *Répéter* et *jusqu'à* sont exécutées au moins une fois et leur exécution est répétée jusqu'à ce que condition soit vraie (tant qu'elle est fausse)

Boucle Répéter jusqu'à : exemple

Un algorithme qui détermine le premier nombre entier N tel que la somme de 1 à N dépasse strictement 100 (**version avec répéter jusqu'à**)

Variables som, i : entier

Debut

 som $\leftarrow$ 0

 i $\leftarrow$ 0

Répéter

 i $\leftarrow$ i+1

 som $\leftarrow$ som+i

Jusqu'à (som > 100)

 Ecrire (" La valeur cherchée est = ", i)

Fin

Choix d'un type de boucle

- Si on peut déterminer le nombre d'itérations avant l'exécution de la boucle, il est plus naturel d'utiliser *la boucle Pour*
- S'il n'est pas possible de connaître le nombre d'itérations avant l'exécution de la boucle, on fera appel à l'une des *boucles TantQue* ou *répéter jusqu'à*
- Pour le choix entre *TantQue* et *jusqu'à* :
 - Si on doit tester la condition de contrôle avant de commencer les instructions de la boucle, on utilisera *TantQue*
 - Si la valeur de la condition de contrôle dépend d'une première exécution des instructions de la boucle, on utilisera *répéter jusqu'à*

ALGORITHMIQUE

Les tableaux

Exemple introductif

- Supposons qu'on veut conserver les notes d'une classe de 30 étudiants pour extraire quelques informations. Par exemple : calcul du nombre d'étudiants ayant une note supérieure à 10
- Le seul moyen dont nous disposons actuellement consiste à déclarer 30 variables, par exemple **N1**, ..., **N30**. Après 30 instructions lire, on doit écrire 30 instructions Si pour faire le calcul

nbre ← 0

Si (N1 >10) alors nbre ←nbre+1 FinSi

....

Si (N30>10) alors nbre ←nbre+1 FinSi

c'est lourd à écrire

- Heureusement, les langages de programmation offrent la possibilité de rassembler toutes ces variables dans **une seule structure de donnée** appelée **tableau**

Tableaux

- Un **tableau** est un ensemble d'éléments de même type désignés par un identificateur unique
- Une variable entière nommée **indice** permet d'indiquer la position d'un élément donné au sein du tableau et de déterminer sa valeur
- La **déclaration** d'un tableau s'effectue en précisant le **type** de ses éléments et sa **dimension** (le nombre de ses éléments)
 - En pseudo code :
variable **tableau** identificateur[**dimension**] : **type**
 - Exemple :
variable **tableau** notes[**30**] : **réel**
- On peut définir des tableaux de tous types : tableaux d'entiers, de réels, de caractères, de booléens, de chaînes de caractères, ...

Tableaux : remarques

- L'accès à un élément du tableau se fait au moyen de l'indice. Par exemple, **notes[i]** donne la valeur de l'élément i du tableau notes
- Il est possible de déclarer un tableau sans préciser au départ sa dimension. Cette précision est faite ultérieurement
 - Par exemple, quand on déclare un tableau comme paramètre d'une procédure, on peut ne préciser sa dimension qu'au moment de l'appel
 - En tous cas, un tableau est inutilisable tant qu'on n'a pas précisé le nombre de ses éléments
- Un grand avantage des tableaux est qu'on peut traiter les données qui y sont stockées de façon simple en utilisant des boucles

Tableaux : exemple

- Calculer le nombre d'étudiants ayant une note supérieure ou égale à 10 en utilisant un tableau à une dimension. Le nombre d'étudiants ainsi que les notes sont à saisir au clavier. Le nombre maximum d'étudiants étant 60.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Tableau à deux dimensions

Déclaration:

- En pseudo code :

Variable **Tableau** identificateur [**val1**, **val2**] : type

- Exemple :

Variable : Tableau T [3,2] : entier

T est une variable de type tableau d'entiers à deux dimensions.

La première dimension a une taille égale à 3 et la deuxième a une taille égale à 2

T peut être vue comme une matrice à 3 lignes et 2 colonnes.

Tableau à deux dimensions

- Soit $T[n, m]$ un tableau d'entiers:
 - T contient $n * m$ cases.
 - Ex : Dans l'exemple précédent, T contient 6 cases.
- $T(i,j)$ désigne la case se trouvant à
 - La ligne i
 - La colonne j

Tableau à deux dimensions

- Lecture : tout comme le tableau à 1 dimension, il faut faire la lecture case par case
- Soit $T[n,m]$

Pour $i = 1$ à n

Pour $j = 1$ à m

Lire($T(i,j)$)

Fin Pour

Fin Pour

Tableau à 2 dimensions

- Soit T la matrice à 2 lignes et 3 colonnes suivantes:

3	4	1
-1	6	0

- Que vont afficher les instructions suivantes?

```
Pour i = 1 à 3
  Pour j = 1 à 2
    Ecrire (T(j,i))
  Fin Pour
Fin Pour
```

```
Pour i = 1 à 2
  Pour j = 1 à 3
    Ecrire (T(i,j))
  Fin Pour
Fin Pour
```

ALGORITHMIQUE

Fonctions et procédures

Fonctions et procédures

- Certains problèmes conduisent à des programmes longs, difficiles à écrire et à comprendre. On les découpe en des parties appelées **sous-programmes** ou **modules**
- Les **fonctions** et les **procédures** sont des modules (groupe d'instructions) indépendants désignés par un nom. Elles ont plusieurs **intérêts** :
 - permettent de "**factoriser**" les programmes, càd de mettre en commun les parties qui se répètent
 - permettent une **structuration** et une **meilleure lisibilité** des programmes
 - **facilitent la maintenance** du code (il suffit de modifier une seule fois)
 - ces procédures et fonctions peuvent éventuellement être **réutilisées** dans d'autres programmes

Fonctions

- Le **rôle** d'une fonction en programmation est similaire à celui d'une fonction en mathématique : elle **retourne un résultat à partir des valeurs des paramètres**
- Une fonction s'écrit en dehors du programme principal sous la forme :

Fonction nom_fonction (paramètres et leurs types) : type_fonction

Instructions constituant le corps de la fonction

retourner ...

FinFonction

- Pour le choix d'un nom de fonction il faut respecter les mêmes règles que celles pour les noms de variables
- type_fonction est le type du résultat retourné
- L'instruction **retourner** sert à retourner la valeur du résultat

Fonctions : exemples

- La fonction SommeCarre suivante calcule la somme des carrées de deux réels x et y :

.....
.....
.....
.....
.....

- La fonction Pair suivante détermine si un nombre est pair :

.....
.....
.....

Utilisation des fonctions

- L'utilisation d'une fonction se fera par simple écriture de son nom dans le programme principale. Le résultat étant une valeur, devra être affecté ou être utilisé dans une expression, une écriture, ...
- **Exepmle : Algorithme exepmleAppelFonction**
.....
.....
.....
.....
.....
- Lors de l'appel Pair(3) le **paramètre formel** n est remplacé par le **paramètre effectif** 3

Procédures

- Dans certains cas, on peut avoir besoin de répéter une tâche dans plusieurs endroits du programme, mais que dans cette tâche on ne calcule pas de résultats ou qu'on calcule plusieurs résultats à la fois
- Dans ces cas on ne peut pas utiliser une fonction, on utilise une **procédure**
- Une **procédure** est un sous-programme semblable à une fonction mais qui **ne retourne rien**
- Une procédure s'écrit en dehors du programme principal sous la forme :

Procédure nom_procédure (paramètres et leurs types)

Instructions constituant le corps de la procédure

FinProcédure

- Remarque : une procédure peut ne pas avoir de paramètres

Appel d'une procédure

- L'appel d'une procédure, se fait dans le programme principal ou dans une autre procédure par une instruction indiquant le nom de la procédure :

Procédure exemple_proc (...)

...

FinProcédure

Algorithme exepmleAppelProcédure

Début

 exemple_proc (...)

...

Fin

- Remarque : contrairement à l'appel d'une fonction, on ne peut pas affecter la procédure appelée ou l'utiliser dans une expression. L'appel d'une procédure est une instruction autonome

Paramètres d'une procédure

- Les paramètres servent à échanger des données entre le programme principale (ou la procédure appelante) et la procédure appelée
- Les paramètres placés dans la déclaration d'une procédure sont appelés **paramètres formels**. Ces paramètres peuvent prendre toutes les valeurs possibles mais ils sont abstraits (n'existent pas réellement)
- Les paramètres placés dans l'appel d'une procédure sont appelés **paramètres effectifs**. ils contiennent les valeurs pour effectuer le traitement
- Le nombre de paramètres effectifs doit être égal au nombre de paramètres formels. L'ordre et le type des paramètres doivent correspondre

Transmission des paramètres

Il existe deux modes de transmission de paramètres dans les langages de programmation :

- **La transmission par valeur** : les valeurs des paramètres effectifs sont affectées aux paramètres formels correspondants au moment de l'appel de la procédure. Dans ce mode le paramètre effectif ne subit aucune modification
- **La transmission par adresse (ou par référence)** : les adresses des paramètres effectifs sont transmises à la procédure appelante. Dans ce mode, le paramètre effectif subit les mêmes modifications que le paramètre formel lors de l'exécution de la procédure
 - **Remarque** : le paramètre effectif doit être une variable (et non une valeur) lorsqu'il s'agit d'une transmission par adresse
- En pseudo-code, les paramètres passés par adresse sont précédées du mot clé **Var**

Transmission des paramètres : exemples

Procédure incrementer1 (**x** : entier, **Var y** : entier)

$x \leftarrow x+1$

$y \leftarrow y+1$

FinProcédure

Algorithme Test_incrementer1

variables n, m : entier

Début

$n \leftarrow 3$

$m \leftarrow 3$

incrementer1(n, m)

écrire (" n= ", n, " et m= ", m)

Fin

résultat :

n=3 et m=4

Remarque : l'instruction $x \leftarrow x+1$ n'a pas de sens avec un passage par valeur

Transmission par valeur, par adresse : exemples

Procédure qui calcule la somme et le produit de deux entiers :

```
.....  
.....  
.....  
.....  
.....
```

Procédure qui échange le contenu de deux variables :

```
.....  
.....  
.....  
.....  
.....
```

Variables locales et globales (1)

- On peut manipuler 2 types de variables dans un module (procédure ou fonction) : des **variables locales** et des **variables globales**. Elles se distinguent par ce qu'on appelle leur **portée** (leur "champ de définition", leur "durée de vie")
- Une **variable locale** n'est connue qu'à l'intérieur du module ou elle a été définie. Elle est créée à l'appel du module et détruite à la fin de son exécution
- Une **variable globale** est connue par l'ensemble des modules et le programme principale. Elle est définie durant toute l'application et peut être utilisée et modifiée par les différents modules du programme

Variables locales et globales (2)

- La manière de distinguer la déclaration des variables locales et globales diffère selon le langage
 - En général, les variables déclarées à l'intérieur d'une fonction ou procédure sont considérées comme variables locales
- En pseudo-code, on va adopter cette règle pour les variables locales et on déclarera les variables globales dans le programme principal
- **Conseil :** Il faut utiliser autant que possible des variables locales plutôt que des variables globales. Ceci permet d'économiser la mémoire et d'assurer l'indépendance de la procédure ou de la fonction

Tableaux : saisie et affichage

- Procédures qui permettent de saisir et d'afficher les éléments d'un tableau :

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Tableaux : exemples d'appel

- Algorithme principal où on fait l'appel des procédures SaisieTab et AfficheTab :

.....
.....
.....
.....
.....
.....
.....
.....
.....
.....

Exemples : lecture d'une matrice

- Procédure qui permet de saisir les éléments d'une matrice :

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Exemples : affichage d'une matrice

- Procédure qui permet d'afficher les éléments d'une matrice :

```
.....  
.....  
.....  
.....  
.....  
.....  
.....  
.....  
.....  
.....  
.....
```

Exemples : somme de deux matrices

- Procédure qui calcule la somme de deux matrices :

.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....

Appel des procédures définies sur les matrices

Exemple d'algorithme principal où on fait l'appel des procédures définies précédemment pour la saisie, l'affichage et la somme des matrices :

.....
.....
.....
.....
.....
.....
.....
.....
.....
.....

Tableaux : 2 problèmes classiques

- **Recherche d'un élément dans un tableau**
 - Recherche séquentielle
 - Recherche dichotomique
- **Tri d'un tableau**
 - Tri par sélection
 - Tri par insertion
 - Tri à bulle
 - Tri rapide